



PRÉ-RENTRÉE 2026-2027

Séance 3 - Fiche enseignant

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Corrigé, différenciation et pilotage

Objectifs

- distinguer paramètre et argument ;
- distinguer print, return et None ;
- comprendre la portée locale ;
- documenter une fonction ;
- écrire des assertions de test ;

Déroulé minute par minute

- 0-10 rituel ;
- 10-30 paramètres/arguments ;
- 30-48 return/None ;
- 48-65 contrats ;
- 65-70 pause ;
- 70-100 module de fonctions ;
- 100-112 tests/débogage ;
- 112-120 revue/exit ;

Rituel prêt à l'emploi

Question 1

Que vaut `r` après l'exécution ?

```
def double(x):  
    print(2 * x)  
  
r = double(5)
```

Réponse attendue : Le programme affiche 10, mais `r` vaut `None` car la fonction ne contient pas de `return`.

Question 2

Dans `def aire_disque(rayon):`, distinguer le paramètre, l'argument dans `aire_disque(3)` et la valeur renvoyée.

Réponse attendue : Paramètre : `rayon` ; argument : `3` ; valeur renvoyée : celle placée après `return`.

Notions de cours

```
def carre(n: int) -> int:
    """Renvoie le carré de n."""
    return n * n
```

- `n` est un paramètre ;
- dans `carre(5)`, `5` est l'argument ;
- `return` termine la fonction et transmet une valeur ;
- `print` affiche mais ne renvoie pas cette valeur ;
- sans `return`, Python renvoie `None` ;
- les variables créées dans la fonction sont locales.

Contrat

- précondition : ce qui doit être vrai avant l'appel ;
- postcondition : ce qui est garanti après l'appel ;
- `assert` peut vérifier une condition ;
- un jeu de tests ne prouve pas l'absence de bugs, mais il réduit le risque.

Activité commune - prédire les retours

```
def f(x):
    print(x + 1)

resultat = f(4)
```

1. Qu'est-ce qui est affiché ?
2. Que vaut `resultat` ?
3. Comment modifier `f` pour renvoyer 5 ?

Conception de tests

Pour une fonction `maximum_deux(a, b)`, proposer :

- un cas ordinaire ;
- un cas où `a == b` ;
- un cas avec deux valeurs négatives.

Parcours Fondations - Ahmad

1. Écrire `est_pair(n)`.
2. Écrire `maximum_deux(a, b)`.
3. Écrire `compte_occurrences(valeurs, cible)`.
4. Ajouter une docstring à chaque fonction.
5. Écrire deux assertions par fonction.

Parcours Fiabilisation - Ahmed

1. Écrire une précondition pour une fonction de moyenne.
2. Choisir le comportement sur liste vide : assertion ou `None`, et le justifier.
3. Écrire des tests limites et invalides.
4. Corriger une fonction qui modifie une variable globale.
5. Lire la documentation de `math.isclose` et l'utiliser pour tester un flottant.

Corrigé essentiel

- le programme affiche `5` et `resultat` vaut `None` ;
- pour renvoyer `5` : `return x + 1` ;
- `est_pair(n)` renvoie `n % 2 == 0` ;
- les tests doivent couvrir égalité, négatifs, zéro et cas limites ;
- une fonction de moyenne doit préciser le comportement sur une liste vide.

Consignes prêtes à dire

- « Avant d'exécuter, écris ce que tu prévois. »
- « Une réponse sans contrôle reste une hypothèse. »
- « Le bug utile est celui que l'on peut reproduire. »
- « Ne change qu'une chose à la fois, puis relance les tests. »
- « Explique le rôle de cette variable sans lire le code mot à mot. »

Points de vigilance

- ne pas transformer l'activité en copie de code projeté ;
- vérifier que les deux élèves alternent pilote et navigateur ;
- ne pas confondre réussite après aide D/E et autonomie ;
- demander un test sur les bornes ;
- conserver le fichier final et le journal des erreurs.

Indicateurs de réussite

Élève	Prévision exacte	Code exécutable	Tests pertinents	Explication	Aide maximale
Ahmad BELDI					
Ahmed BENHADJ SALEM					